



Editorial

Tasks, test data and solutions were prepared by: Fran Babić, Toni Brajko, Nikola Dmitrović, Sofija Velkovska and Jakov Celin.

Implementation examples are given in attached source code files.

Task Rima

Prepared by: Sofija Velkovska

Necessary skills: strings, loops

First of all, we need to find a way to determine if two lines rhyme. We will consider the last words of each line, let's call them a and b . Before anything else, we have to check if both strings are of length at least k . If either a or b are made of less than k characters, the two lines don't rhyme, and we shouldn't proceed with any further checks. If both strings are long enough, we need to check whether their last k characters match. If we denote the length of a and b with a_len and b_len , respectively, we have to compare the letter at position $a_len - k$ in a with the letter at position $b_len - k$ in b , the letter at position $a_len - k + 1$ in a with the letter at position $b_len - k + 1$ in b etc. up until the letters at positions $a_len - 1$ in a and $b_len - 1$ in b (the last letters in the strings). If in any of these pairs the two letter are not the same, the lines don't rhyme. Otherwise (if all of the pairs are made up of two equal letters), the lines rhyme. We can do this by running a *for* loop.

There are other ways to check whether two lines rhyme. For example, we can use `a[-k:]` and `b[-k:]` in Python, or `a.substr(a_len - k, k)` and `b.substr(b_len - k, k)` in C++, to get the last k letters of the words and then compare them with the `"=="` operator. Note that we still need to check whether the length of both a and b is at least k before performing any of these operations.

Now that we have an algorithm that will tell us whether two lines rhyme, we can use it to check the types of rhyme schemes that appear in each stanza. We want to process the poem stanza by stanza, and in order to do that we can use a *for* loop that goes from the first line to the last with a +4 increment operation (`for (int i = 0; i < n; i = i + 4)`). Let's say we are currently at line i , the stanza in which i is located is made up of lines $i, i + 1, i + 2$ and $i + 3$.

To check whether a certain stanza follows an AABB rhyme scheme, we check whether lines i and $i + 1$, and lines $i + 2$ and $i + 3$ rhyme using the previously explained procedure. If both of the pairs of lines rhyme, the stanza follows an AABB rhyme scheme. We repeat this for the other types of rhyme schemes, only this time we pair up the lines in a different way. For ABAB we pair up i with $i + 2$, and $i + 1$ with $i + 3$, and for ABBA we pair up i with $i + 3$, and $i + 1$ with $i + 2$. We need to keep count of the number of stanzas that follow each rhyme scheme, and we can do that by having three different integer variables, one for every rhyme scheme, that we increase each time a stanza passes the criteria for that particular rhyme scheme.

Task Krugomet

Prepared by: Jakov Celin

Necessary skills: binary lifting

Notice that person i always throws all of their balls to person s_i . Then, person s_i throws those balls (and others they might have received, which we'll ignore for now) to person s_{s_i} , then s_{s_i} throws them to... The statement suggests that for each person i , we should determine where the balls that originally belonged to person i will end up after K steps. Then, we would assign those balls to that person, and finally, we would check who has the most balls and print their list.

All that remains is to find the person who will end up with the balls originally from person i .



For $K = 1$, that person is s_i .

For $K = 2$, that person is s_{s_i} .

For $K = 3$, that person is $s_{s_{s_i}}$.

Let's call the person reached after X consecutive "jumps" from person i — each time following the sequence s — the X -th *crush* of person i . It follows that after K steps, all balls from person i will end up with their K -th crush.

For 14 points, it was enough to simply simulate this process using a for loop. For an additional 26 points, it was necessary to decompose the permutation into cycles and, using K and the cycle lengths, determine all K -th crushes. The problem could be solved for full points in many ways using the structure of the "crush graph," but those solutions are more complex to implement and understand than the official one, so we'll describe only the official solution here, which uses the algorithm known as *binary lifting*.

If, for powers of two, we know the 2^x -th crush of each person, then we can find the K -th crush of any person in $O(\log K)$ time. If we write K in binary (it consists of exactly $\log K + 1$ digits), we only need to make jumps to the 2^x -th crushes where x corresponds to the positions of the 1 bits in the binary representation of K .

What remains is to compute the 2^x -th crushes of every person for $1 \leq x \leq 30$, since $1 \leq K \leq 10^9$. Notice that $2^x = 2^{x-1} + 2^{x-1}$, i.e. the jump to the 2^x -th crush can be represented as a jump to the 2^{x-1} -th crush, followed by another jump to its 2^{x-1} -th crush. If we know all the first crushes, using the previous statement we can compute all the second crushes, then the fourth ones, and so on. The time and memory complexity of the described algorithm is $O(n \log K)$. For details, refer to the official implementation.

Task Harmonija

Prepared by: Jakov Celin

Necessary skills: binary lifting, LCA, matrices, dynamic programming

Notice that from the condition of a harmonic path, at every moment along the path between any two nodes, the difference between the number of red and blue nodes must satisfy ≤ 2 , i.e. $-2 \leq \text{numRedNodes} - \text{numBlueNodes} \leq 2$. This suggests a solution where we transform the path from node A to node B into a sequence, which we then solve using dynamic programming.

Let $dp[i][diff]$ represent the maximum value of the harmonic path from A to the i -th node along the path from A to B, with the current color difference equal to $diff$. We branch into two transitions (choosing whether the i -th node is red or blue) and check whether the color difference remains between -2 and 2 after the choice. This algorithm yields 68 points due to its high time complexity of $O(nq)$.

Full points were obtained by several solutions using various techniques (centroid decomposition, HLD, ...), but we'll describe the simplest one.

Let's denote by AB_{xy} the maximum value of a harmonic path from node A to node B that starts with color difference x and ends with color difference y . Notice that if node B lies on the path between nodes A and D, and node C is the first node on the path from A to D after B, then the following holds: $AB_{xy} + CD_{yz} = AD_{xz}$. This statement allows us to merge the "best" harmonic paths with the same endpoint into longer ones.

The main idea is to, for chains whose lengths are powers of two, store a 5×5 matrix representing which color difference we start with when entering the chain and which we end with when exiting it. We answer a query on the path from A to B by splitting the path into $O(\log n)$ disjoint chains using the standard binary lifting/LCA algorithm and then merging the chains similarly to matrix multiplication, applying the previous property. With careful implementation (e.g., chains from the LCA to B must be merged in reverse order during both query processing and preprocessing), this algorithm achieves full points on the task. For details, refer to the official implementation.



Task Kraljica

Prepared by: Jakov Celin

Necessary skills: 0-1 BFS

For simpler complexity analysis, we will assume that in the worst case $N = M = 1000$, so we'll write $N = M$, even though that might not necessarily hold.

The first and third subtasks could be solved using the standard BFS algorithm, which attempts to expand from the queen's current square to all possible squares the queen can move to. The time complexity of this approach is $O(N^3)$.

The second and fourth subtasks require knowledge of 0-1 BFS, and the fourth one also requires handling portals in addition to the queen's regular moves. The key observation leading to the solution is that for every square, we should also remember from which direction we arrived at that square — that is, our state will not only consist of the row and column of the matrix cell but also of the direction we came from. Now, from a state (**direction**, **cell**), where the direction is a number from 0 to 8 and the cell is represented by its row and column indices, we branch into 8 adjacent cells (if possible) and the portal (if one exists on our square). The transition weights are either 0 or 1, depending on whether we kept the same direction (no additional move — weight 0) or changed it (a new move is needed — weight 1).

The algorithm that computes the shortest path between two nodes of a graph where edge weights are either 0 or 1 is called **0-1 BFS**, and its implementation can be found in the provided code. For simpler implementation, the portal transition state is marked with direction 9, since this makes the implementation identical to that of the second subtask (without portals). The algorithm's complexity is $O(NM)$, but with a large constant factor (each of the $9NM$ states branches into 8 neighbors and sometimes into a portal). For details, see the official implementation.

Task Zagi

Prepared by: Fran Babić

Required knowledge: Game theory, Sprague–Grundy theorem, ad hoc reasoning, recursion memoization

In the case where $a_i \leq 2$, after the first player makes a move (i.e., chooses 1 or 2), the remaining sequences will consist of numbers that are all equal to each other. We can conclude that the game will end in exactly as many moves as there are remaining subsequences, and the winner depends on the parity of that number. The first player can choose a number such that the number of remaining sequences is even. If such a choice exists, the first player will win; otherwise, the second player wins.

For solving the remaining subtasks, we introduce the notation $S(l, r)$, which denotes the *Grundy number* of the state where we have the numbers $a_l, a_{l+1}, \dots, a_r$. We also introduce the notation $\text{mex}(\{b_1, b_2, \dots, b_n\})$, representing the smallest non-negative integer that **does not appear** in the set $\{b_1, b_2, \dots, b_n\}$, and the operation $\cdot \oplus \cdot : \mathbb{N}_0^2 \rightarrow \mathbb{N}_0$, which represents the so-called *binary XOR*.

In any given scenario, using the Sprague–Grundy theorem, the first player wins if and only if $S(l, r) \neq 0$. It remains to show how to compute $S(l, r)$.

When the player on move chooses a number $x \in \{a_l, \dots, a_r\}$, the game “splits” into independent subsequences $(l, p_{x,1} - 1)$, $(p_{x,1} + 1, p_{x,2} - 1)$, $\dots$, $(p_{x,k_x} + 1, r)$, where $p_{x,i}$ are the positions of x within that subsequence. It follows that:

$$S(l, r) = \text{mex}(\{S(l, p_{x,1} - 1) \oplus S(p_{x,1} + 1, p_{x,2} - 1) \oplus \dots \oplus S(p_{x,k_x} + 1, r) : x \in \{a_l, \dots, a_r\}\})$$

If we examine this expression, we notice that for a given (l, r) and for some x from that subsequence, the above expression consists of a prefix and a suffix of the interval (l, r) and of contiguous subsequences located between the occurrences of x . The total number of such subsequences is bounded by nm , where m is the number of distinct elements in the given sequence.



When we take into account all q scenarios, we observe that the number of contiguous subsequences for which we need to compute the corresponding Grundy number is bounded by $O(m(n + q))$. With a careful implementation of this algorithm, we can achieve the full score for this problem. For more details, refer to the official implementation.